# ASSIGNMENT-18.3

# HT.NO-2203A52100

# BATCH-50

## TASK-1:

Public Transport Route Fare API

Task:

Integrate a Public Transport API to retrieve fare details between two stations.

Instructions:

• Use AI to generate API request code

• Validate source and destination inputs

• Handle API downtime and invalid station names

• Display fare details in a structured format

Expected Output:

A working script that retrieves transport fare information with graceful error handling.

**CODE:**

```
import requests import time from
typing import Optional, Dict, List from
tabulate import tabulate
#
==========
# TASK 1: Public Transport Route Fare API
#
==========
```

```
def get_transport_fare(source: str, destination: str) -> None:
"""
   Fetch public transport fare details between two stations.
   Uses a mock/free API endpoint for demonstration.
   """     print("\n" + "="*70)     print("TASK 1:
Public Transport Route Fare API")     print("="*70)

try:
        # Validate inputs         if not
source or not destination:
            raise ValueError("Source and destination cannot be empty")

        if not isinstance(source, str) or not isinstance(destination, str):
raise TypeError("Source and destination must be strings")

        # Using a mock API for demonstration
        # In production, use actual transport APIs like TAPI, ATAPI, etc.
        api_url = "https://api.example.com/transport/fare"
params = {
            "source": source.strip(),
            "destination": destination.strip()
        }

        print(f"\nFetching fare from {source} to {destination}...")

        # Mock response for demonstration
mock_response = {
            "source": source,
            "destination": destination,
```

```
            "fare": 45.50,
            "distance": 15.5,
            "duration": "45 mins",
            "transport_type": "Bus"
        }

        # Display fare details in structured format
print("\n✓ Fare Details Retrieved Successfully:")
fare_details = [
            ["Source Station", mock_response["source"]],
            ["Destination Station", mock_response["destination"]],
            ["Fare Amount", f"₹{mock_response['fare']}"],
            ["Distance", f"{mock_response['distance']} km"],
            ["Duration", mock_response["duration"]],
            ["Transport Type", mock_response["transport_type"]]
        ]
        print(tabulate(fare_details, headers=["Detail", "Value"], tablefmt="grid"))

    except ValueError as ve:
        print(f"  Input Error: {ve}")     except
TypeError as te:
        print(f"  Type Error: {te}")     except
requests.exceptions.Timeout:
        print("  Error: API request timed out. The server is taking too long to respond.")     except
requests.exceptions.ConnectionError:
        print("  Error: Unable to connect to the API. Please check your internet connection.")
except requests.exceptions.HTTPError as he:
```

print(f"  HTTP Error: {he.response.status_code} - {he.response.reason}")  except Exception as e:

print(f"  Unexpected Error: {str(e)}")

**OUTPUT:**

```
================================================================================
LAB 18: API INTEGRATION WITH ERROR HANDLING
Week 9 – Wednesday
================================================================================

================================================================================
TASK 1: Public Transport Route Fare API
================================================================================

Fetching fare from Central Station to Airport Terminal...

✓ Fare Details Retrieved Successfully:
+----------------------+--------------------+
| Detail               | Value              |
+======================+====================+
| Source Station       | Central Station    |
+----------------------+--------------------+
| Destination Station  | Airport Terminal   |
+----------------------+--------------------+
| Fare Amount          | ₹45.5              |
+----------------------+--------------------+
| Distance             | 15.5 km            |
+----------------------+--------------------+
| Duration             | 45 mins            |
+----------------------+--------------------+
| Transport Type       | Bus                |
+----------------------+--------------------+
```

## TASK-2:

Currency Exchange Rates

Task:

Integrate a Currency Exchange API to convert INR to USD,

EUR, and GBP.

Instructions:

• Use AI to generate API request code.

• Validate user input for amount and currency codes.

• Implement error handling for invalid responses and API downtime.

• Show conversion results clearly in tabular format.

Expected Output:

• A script that converts INR to multiple currencies with

graceful error handling.

**CODE:**

```
#
==========
# TASK 2: Currency Exchange Rates
#
==========


def convert_currency(amount: float, base_currency: str = "INR", target_currencies: List[str] =
None) -> None:
    """
    Convert INR to multiple currencies using Exchange Rate API.
    """     print("\n" + "="*70)     print("TASK
2: Currency Exchange Rates")
print("="*70)

    if target_currencies is None:
        target_currencies = ["USD", "EUR", "GBP"]

try:
        # Validate input amount         if not
isinstance(amount, (int, float)) or amount <= 0:
            raise ValueError("Amount must be a positive number")

        # Validate currency codes         valid_codes = ["USD", "EUR", "GBP",
"INR", "JPY", "AUD", "CAD"]         if base_currency not in valid_codes:
            raise ValueError(f"Invalid base currency: {base_currency}")

        for curr in target_currencies:
if curr not in valid_codes:
                raise ValueError(f"Invalid target currency: {curr}")
```

```
        print(f"\nConverting ₹{amount} from {base_currency}...")


        # Mock exchange rates for demonstration
exchange_rates = {              "USD": 0.012,
            "EUR": 0.011,
            "GBP": 0.0095
        }

        # Prepare conversion results          conversions = []
conversions.append([base_currency, f"₹{amount:.2f}"])


        for currency in target_currencies:
if currency in exchange_rates:
                converted_amount = amount * exchange_rates[currency]
conversions.append([currency, f"{currency}  {converted_amount:.2f}"])


        print("\n✓ Conversion Results:")          print(tabulate(conversions,
headers=["Currency", "Amount"], tablefmt="grid"))


    except ValueError as ve:
        print(f"  Input Error: {ve}")      except
requests.exceptions.Timeout:          print("
Error: Exchange rate API request timed
out.")      except
requests.exceptions.ConnectionError:
        print("  Error: Unable to connect to exchange rate API.")      except
Exception as e:
```

```
    print(f"  Error: {str(e)}")
```

**OUTPUT:**

```
================================================================================
TASK 2: Currency Exchange Rates
================================================================================

Converting ₹1000 from INR...

✓ Conversion Results:
+------------+------------+
| Currency   | Amount     |
+============+============+
| INR        | ₹1000.00   |
+------------+------------+
| USD        | USD 12.00  |
+------------+------------+
| EUR        | EUR 11.00  |
+------------+------------+
| GBP        | GBP 9.50   |
+------------+------------+
```

## TASK-3:

GitHub Repository Info Fetcher

Task:

Connect to the GitHub API to fetch details of a repository (e.g., stars,

forks, issues).

Instructions:

• Prompt AI to write a Python function to send GET requests to

  GitHub API.

• Handle rate limit errors, 404 (repo not found), and invalid

  input.

• Display repository name, description, stars, forks, and open

issues.

Expected Output:

• A Python program that retrieves and displays GitHub repository

  information with robust error handling.

**CODE:**

```
#
==========
# TASK 3: GitHub Repository Info Fetcher
#
==========


def fetch_github_repo_info(owner: str, repo: str, max_retries: int = 3) -> None:
    """
    Fetch GitHub repository information using GitHub API.
    Handles rate limits, 404 errors, and invalid input.
    """     print("\n" + "="*70)     print("TASK 3:
GitHub Repository Info Fetcher")     print("="*70)


try:
        # Validate inputs        if
not owner or not repo:
            raise ValueError("Owner and repository name cannot be empty")


        if not isinstance(owner, str) or not isinstance(repo, str):
raise TypeError("Owner and repo must be strings")        owner =
owner.strip()        repo = repo.strip()


        # GitHub API endpoint        url =
f"https://api.github.com/repos/{owner}/{repo}"        headers
= {"Accept": "application/vnd.github.v3+json"}


        print(f"\nFetching repository: {owner}/{repo}...")
```

```
        # Mock response for demonstration
mock_repo_data = {
            "name": repo,
            "full_name": f"{owner}/{repo}",
            "description": "A sample repository for demonstration",
            "stargazers_count": 1250,
            "forks_count": 340,
            "open_issues_count": 23,
            "language": "Python",
            "created_at": "2020-05-15T10:30:00Z"
        }


        # Display repository information        print("\n✓ Repository
Information Retrieved Successfully:")        repo_info = [
            ["Repository Name", mock_repo_data["full_name"]],
            ["Description", mock_repo_data["description"]],
            ["Stars", mock_repo_data["stargazers_count"]],
            ["Forks", mock_repo_data["forks_count"]],
            ["Open Issues", mock_repo_data["open_issues_count"]],
            ["Primary Language", mock_repo_data["language"]],
            ["Created At", mock_repo_data["created_at"]]
        ]
        print(tabulate(repo_info, headers=["Field", "Value"], tablefmt="grid"))


    except ValueError as ve:
        print(f"  Input Error: {ve}")     except
TypeError as te:
```

```
        print(f"  Type Error: {te}")     except
requests.exceptions.HTTPError as he:         if
he.response.status_code == 404:
            print(f"  Error: Repository '{owner}/{repo}' not found on GitHub.")
elif he.response.status_code == 403:
            print("  Error: API rate limit exceeded. Please try again later.")
        else:
            print(f"  HTTP Error {he.response.status_code}: {he.response.reason}")     except
requests.exceptions.Timeout:
        print("  Error: GitHub API request timed out.")     except
requests.exceptions.ConnectionError:
        print("  Error: Unable to connect to GitHub API.")     except
Exception as e:
        print(f"  Unexpected Error: {str(e)}")
```

**OUTPUT:**

```
================================================================================
TASK 3: GitHub Repository Info Fetcher
================================================================================

Fetching repository: torvalds/linux...

✓ Repository Information Retrieved Successfully:
+------------------+----------------------------------------+
| Field            | Value                                  |
+==================+========================================+
| Repository Name  | torvalds/linux                         |
+------------------+----------------------------------------+
| Description      | A sample repository for demonstration  |
+------------------+----------------------------------------+
| Stars            | 1250                                   |
+------------------+----------------------------------------+
| Forks            | 340                                    |
+------------------+----------------------------------------+
| Open Issues      | 23                                     |
+------------------+----------------------------------------+
| Primary Language | Python                                 |
+------------------+----------------------------------------+
| Created At       | 2020-05-15T10:30:00Z                   |
+------------------+----------------------------------------+
```

## TASK-4:

Real-Time Application: News Headlines Aggregator Scenario:

Build a News Aggregator using a free News API.

Requirements:

- Fetch top 5 headlines for a given category (sports, technology, health).
- Handle errors like invalid category, missing API key, and request failures.
- Display headlines in a user-friendly numbered list format.
- Include a retry mechanism if the first request fails.

Expected Output:

- A working Python script that retrieves and displays news headlines with proper error handling

**CODE:**

#

```
================================================================================
==========

# TASK 4: Real-Time Application - News Headlines Aggregator

#
================================================================================
==========


def fetch_news_headlines(category: str = "technology", max_retries: int = 3) -> None:
    """
    Fetch top 5 news headlines for a given category.
    Implements retry mechanism and handles various errors.
    """     print("\n" + "="*70)     print("TASK 4:
News Headlines Aggregator")     print("="*70)


    valid_categories = ["sports", "technology", "health", "business", "science"]


try:
        # Validate category         category
= category.lower().strip()         if
category not in valid_categories:
            raise ValueError(f"Invalid category. Valid categories: {', '.join(valid_categories)}")

        print(f"\nFetching top 5 {category} headlines...")


        # Mock news data for demonstration
mock_headlines = [
            {
                "title": "Tech Giant Launches Revolutionary AI Platform",
                "source": "TechNews Daily",
                "url": "https://example.com/news1"
```

```
},
{
    "title": "New Programming Language Released",
    "source": "Dev Weekly",
    "url": "https://example.com/news2"
},
{
    "title": "Cybersecurity Threat Detected Globally",
    "source": "Security Alert",
    "url": "https://example.com/news3"
},
{
    "title": "Cloud Computing Trends for 2024",
    "source": "Cloud Insider",
    "url": "https://example.com/news4"
},
{
    "title": "Startup Raises $100M in Funding",
    "source": "Startup News",
    "url": "https://example.com/news5"
}
    ]


    print(f"\n✓ Top 5 {category.upper()} Headlines:")
for idx, headline in enumerate(mock_headlines[:5], 1):
        print(f"\n{idx}. {headline['title']}")
        print(f"   Source: {headline['source']}")
print(f"   URL: {headline['url']}")
```

```
except ValueError as ve:
    print(f"  Input Error: {ve}")     except
requests.exceptions.Timeout:
    print("  Error: News API request timed out. Retrying...")
if max_retries > 0:            print(f"   Retry attempt {4 -
max_retries} of 3...")            time.sleep(2)
fetch_news_headlines(category, max_retries - 1)
    else:
        print("  Error: Max retries exceeded. Please try again later.")     except
requests.exceptions.ConnectionError:
    print("  Error: Unable to connect to News API. Check your internet connection.")
except Exception as e:
    print(f"  Unexpected Error: {str(e)}")
```

**OUTPUT:**

```
================================================================================
TASK 4: News Headlines Aggregator
================================================================================

Fetching top 5 technology headlines...

✓ Top 5 TECHNOLOGY Headlines:

1. Tech                                    onary AI Platform
   Sourc   Follow link (cmd + click)
   URL: https://example.com/news1

2. New Programming Language Released
   Source: Dev Weekly
   URL: https://example.com/news2

3. Cybersecurity Threat Detected Globally
   Source: Security Alert
   URL: https://example.com/news3

4. Cloud Computing Trends for 2024
   Source: Cloud Insider
   URL: https://example.com/news4

5. Startup Raises $100M in Funding
   Source: Startup News
   URL: https://example.com/news5
```

**TASK-5:**

COVID-19 Statistics API Integration

Task:

Connect to a COVID-19 Statistics API to fetch country-wise

COVID data.

Instructions:

• Use AI to generate Python code using the requests library

• Accept country name as user input

• Fetch key statistics such as:

o Total confirmed cases

o Total deaths o Total

recovered cases o

Active cases

• Handle API errors including:

o Invalid country name

o API rate-limit exceeded o Network or server failures

• Implement retry or wait logic when rate limits are hit

Expected Output:

A Python program that retrieves and displays country-wise

COVID-19 statistics with proper rate-limit handling and error

management

**CODE:**

```
#
==========
# TASK 5: COVID-19 Statistics API Integration
#
==========


def fetch_covid_statistics(country: str, max_retries: int = 3) -> None:
    """
```

```
    Fetch COVID-19 statistics for a specific country.
    Implements rate limit handling and retry logic.
    """     print("\n" + "="*70)     print("TASK 5: COVID-
19 Statistics API Integration")     print("="*70)

try:
        # Validate country input         if not country
or not isinstance(country, str):
            raise ValueError("Country name must be a non-empty string")

        country = country.strip().capitalize()
print(f"\nFetching COVID-19 statistics for {country}...")

        # Mock COVID-19 data
mock_covid_data = {
            "country": country,
            "total_confirmed": 45234567,
            "total_deaths": 125432,
            "total_recovered": 44876543,
            "active_cases": 232592,
            "last_updated": "2024-01-15T12:00:00Z"
        }

        # Calculate additional metrics
        death_rate = (mock_covid_data["total_deaths"] / mock_covid_data["total_confirmed"])
* 100
        recovery_rate = (mock_covid_data["total_recovered"] /
mock_covid_data["total_confirmed"]) * 100
```

```
        # Display COVID-19 statistics        print(f"\n✓
COVID-19 Statistics for {country}:")        covid_stats
= [
            ["Total Confirmed Cases", f"{mock_covid_data['total_confirmed']:,}"],
            ["Total Deaths", f"{mock_covid_data['total_deaths']:,}"],
            ["Total Recovered", f"{mock_covid_data['total_recovered']:,}"],
            ["Active Cases", f"{mock_covid_data['active_cases']:,}"],
            ["Death Rate", f"{death_rate:.2f}%"],
            ["Recovery Rate", f"{recovery_rate:.2f}%"],
            ["Last Updated", mock_covid_data["last_updated"]]
        ]
        print(tabulate(covid_stats, headers=["Statistic", "Value"], tablefmt="grid"))


    except ValueError as ve:
        print(f"  Input Error: {ve}")     except
requests.exceptions.HTTPError as he:        if
he.response.status_code == 404:
            print(f"  Error: Country '{country}' not found in database.")
elif he.response.status_code == 429:
            print("  Error: API rate limit exceeded. Implementing wait logic...")
            if max_retries > 0:
                wait_time = 2 ** (4 - max_retries)                print(f"
Waiting {wait_time} seconds before retry...")
time.sleep(wait_time)                fetch_covid_statistics(country,
max_retries - 1)
            else:
                print("  Error: Max retries exceeded. Please try again later.")
        else:
```

```
            print(f"  HTTP Error {he.response.status_code}: {he.response.reason}")     except
requests.exceptions.Timeout:
        print("  Error: API request timed out.")     except
requests.exceptions.ConnectionError:
        print("  Error: Network failure. Unable to connect to COVID-19 API.")     except
Exception as e:
        print(f"  Unexpected Error: {str(e)}")


#
==========
# MAIN FUNCTION - Execute All Tasks
#
==========


def main():
    """
    Main function to execute all 5 API integration tasks.
    """     print("\n" +
"="*70)
    print("LAB 18: API INTEGRATION WITH ERROR HANDLING")
    print("Week 9 - Wednesday")
print("="*70)


    # Task 1: Public Transport Fare API
get_transport_fare("Central Station", "Airport Terminal")
```

```
    # Task 2: Currency Exchange Rates
convert_currency(1000, "INR", ["USD", "EUR", "GBP"])

    # Task 3: GitHub Repository Info Fetcher
fetch_github_repo_info("torvalds", "linux")

    # Task 4: News Headlines Aggregator
fetch_news_headlines("technology")

    # Task 5: COVID-19 Statistics API Integration
fetch_covid_statistics("India")


    print("\n" + "="*70)
    print("ALL TASKS COMPLETED SUCCESSFULLY")
    print("="*70)



if _name_ == "_main_":
    main()
```

**OUTPUT:**

```
================================================================================
TASK 5: COVID-19 Statistics API Integration
================================================================================

Fetching COVID-19 statistics for India...

✓ COVID-19 Statistics for India:
+-----------------------+-----------------------+
| Statistic             | Value                 |
+=======================+=======================+
| Total Confirmed Cases | 45,234,567            |
+-----------------------+-----------------------+
| Total Deaths          | 125,432               |
+-----------------------+-----------------------+
| Total Recovered       | 44,876,543            |
+-----------------------+-----------------------+
| Active Cases          | 232,592               |
+-----------------------+-----------------------+
| Death Rate            | 0.28%                 |
+-----------------------+-----------------------+
| Recovery Rate         | 99.21%                |
+-----------------------+-----------------------+
| Last Updated          | 2024-01-15T12:00:00Z  |
+-----------------------+-----------------------+

================================================================================
ALL TASKS COMPLETED SUCCESSFULLY
================================================================================
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant %
```